# Project 15.1 — FeedbackLens-Customer-Feedback-Batch-Analyzer

**A practice twin of Project 15 — MeetingMind.** Same skills — several narrow extractors run in parallel ( `RunnableParallel` ), each with its own typed output, folded together by a deterministic compiler — on a new real-world job. **There are no steps here on purpose.** Read the story, build it yourself, and open the hints or solution only if you get stuck.

**Type:** Multi-agent (3 parallel extractors, one compiler) · **Stack:** Python, LangChain (LCEL, `RunnableParallel` ), OpenAI SDK, Pydantic · **Level:** Intermediate

> **Stuck, or want to check your work?** Full hints · Complete solution

## The Story — what you are going to build

A product team collects a pile of customer reviews and nobody has time to read them all. They want three things out of the batch: **what people love**, **what people complain about**, and **the overall mood** in a sentence or two.

You are going to build **FeedbackLens**: paste in a batch of reviews, get back those three things.

- A **praises extractor** returns a typed list of the features customers liked.
- A **complaints extractor** returns a typed list of the problems customers raised.
- A **sentiment summarizer** returns a few plain sentences on the overall mood.
- The three run **in parallel** (none depends on the others), and a deterministic **compiler** folds them into one `FeedbackReport` .

**The one lesson of this project:** same as MeetingMind — "analyze this pile of text" is usually *several narrow extractions*, not one giant prompt. Each extractor gets its own job and its own typed shape; the independent ones run at once; a plain-Python compiler assembles the report.

## How it works, from reviews to a report (picture the whole machine)

The batch of reviews (one text blob) goes into a `RunnableParallel` that runs all three chains at the same time with the same input. Two return typed lists (praises, complaints) via `with_structured_output` ; the summarizer returns text via `StrOutputParser` . The parallel step

hands back one dict — `{"summary": ..., "praises": ..., "complaints": ...}` — and the deterministic **compiler** turns it into a `FeedbackReport`, which a formatter prints for a human.

That's the whole machine: **reviews → RunnableParallel(3 extractors) → compile → one report (praises, complaints, mood).**

## What you are building (the pieces)

- Typed shapes: `Praises(items)`, `Complaints(items)`, `FeedbackReport(summary, praises, complaints)`.
- Three chains: `build_praises_chain`, `build_complaints_chain`, `build_summary_chain`.
- A deterministic `compile_report` / `format_report`.
- A pipeline (`RunnableParallel` + `run_analysis(model, reviews)`).

**Inputs:** a batch of review text. **Outputs:** one report — praises, complaints, and a mood summary. **Rules:** one specialist per job; typed output for the lists, text for the summary; run the three in parallel; compile deterministically.

## A real example to build and test against

- Reviews praising the battery and screen, complaining about the price and setup → a report that lists "battery", "screen" under praises, "price", "setup" under complaints, and a one-line mood summary.
- An empty-ish batch with nothing to extract → empty lists, a neutral summary (and your compiler/formatter still produce a valid report).

## How to practice this

1. Read this story. **Don't open the solution yet.** (Review Project 15 / Doc05's `RunnableParallel` if hazy.)
2. Build the compiler and formatter **first** and test them with hand-made parts — no model. That's the part you can fully pin down.
3. Build the three chains (two `with_structured_output`, one `StrOutputParser`), then wire them into a `RunnableParallel`.
4. Test it on the real example: a batch with clear praises and complaints.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

## Why this project is useful (read this at the end)

"Summarize and categorize a pile of feedback" is a job every product, support, and research team has — reviews here, but the same shape fits survey responses, support tickets, interview notes, app-store comments. The skill is the structure: narrow specialists, typed outputs, parallel where independent, deterministic compile. Build FeedbackLens and you can turn any pile of text into a clean, structured report.

## Done when

- Three specialist chains, each one job; two typed, one text.
- They run in parallel with `RunnableParallel`.
- The compiler/formatter is deterministic and tested with no model.
- `run_analysis` returns one report with praises, complaints, and a mood summary.
- Your tests pass: the compiler (hand-made parts) and a live run on a real batch.

---

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.